Sistemas Embebidos I

Otoño 2026

Práctica #7: Serial Communication & Scheduling: PWM

María Fernanda Conde Calatayud

Andrea Ceja Lara

Kiria Margalli Chumacero

Jorge Emiliano Sánchez Rodríguez

09/09/26

Índice

Práctica #7: Serial Communication & Scheduling: PWM *2*

Introducción **2**

Descripción de la práctica 2

Objetivo 2

Marco teórico 2

Desarrollo **4**

Materiales 4

Esquemático 4

Desarrollo 5

Resultados **9**

Evidencias **13**

Conclusiones **14**

Práctica #7: Serial Communication & Scheduling: PWM

Introducción

Descripción de la práctica

La práctica consiste en controlar cuatro cargas diferentes de manera simultánea desde una consola serial utilizando la modulación por ancho de pulso (PWM) en un microcontrolador (como la Raspberry Pi Pico 2).

Cada carga se conecta en su propio canal/slice de PWM:

- **LED con resistencia** (GP2, Slice 1, alimentado directamente por el pin de la Pico).
- **Buzzer pasivo** (GP4, Slice 2, alimentado por el pin de la Pico).
- **Servomotor** (GP6, Slice 3, alimentado con una fuente externa de 5 V y con tierra común).
- **Motor DC con MOSFET logic-level y diodo de recirculación** (GP8, Slice 4, alimentado con una fuente externa de 5 V).

A través de comandos por la consola serial (como led, tono, servo, motor y estado), se modifican los parámetros de las señales para evaluar su comportamiento en cuanto a luz, sonido, movimiento y respuesta del sistema.

Objetivo

Controlar de forma concurrente cuatro cargas independientes (un LED, un buzzer, un servomotor y un motor de corriente directa) aplicando los principios de la modulación por ancho de pulso (PWM), configuración de contadores, divisores de reloj, manejo de potencias y procesamiento de comandos por comunicación serial.

Marco teórico

Modulación por Ancho de Pulso (PWM)

Es una técnica para controlar la energía que recibe una carga mediante una señal digital que conmuta entre un nivel alto y uno bajo a una frecuencia fija, variando la fracción de cada ciclo que permanece encendida (Duty Cycle). La señal digital nunca toma valores intermedios; el valor promedio o efecto analógico es interpretado directamente por la carga en función de su inercia o respuesta temporal.

Parámetros Fundamentales del PWM

- **Periodo (T):** Duración de un ciclo completo de la señal.
- **Frecuencia ($f = 1/T$):** Número de ciclos completos por segundo.

- **Ancho de pulso (t_{on}):** Tiempo que la señal permanece en nivel alto durante un periodo.
- **Duty Cycle ($D = t_{on}/T$):** Fracción o porcentaje del periodo en que la señal se encuentra activa.

Funcionamiento Interno en el Microcontrolador (RP2040 / RP2350)

El hardware de PWM del microcontrolador utiliza dos elementos principales por cada slice:

1. **Contador:** Incrementa su valor de 0 hasta un valor máximo denominado TOP a una velocidad dictada por un divisor de reloj (div). Esto define la frecuencia de la señal.
2. **Comparador (Nivel o CC):** Compara el valor del contador para mantener la salida en nivel alto mientras el contador sea menor al valor de level, determinando así el ciclo de trabajo (*Duty Cycle*).

La frecuencia de salida se calcula mediante la fórmula:

$$f_{PWM} = \frac{f_{clk}}{div \cdot (TOP + 1)}$$

Comportamiento según el Tipo de Carga

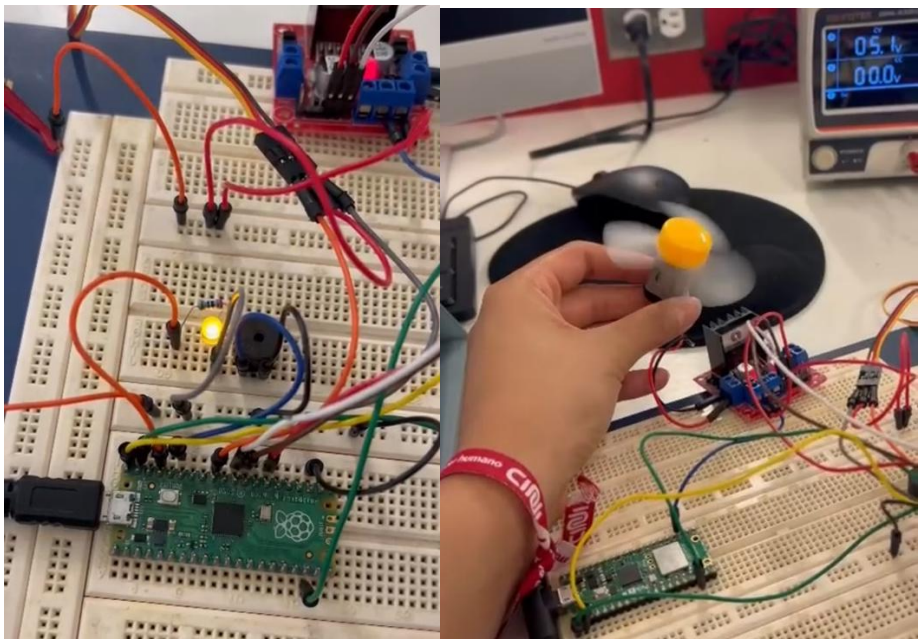
- **LEDs:** La luz percibida por el ojo humano no responde de forma lineal al ciclo de trabajo, sino de manera logarítmica (requiriendo correcciones gamma como $D = B^{2.2}$ para cambios de brillo uniformes). Frecuencias superiores a 1 kHz evitan parpadeos visibles y problemas con cámaras (*rolling shutter*).
- **Buzzers (Zumbadores):** En los buzzers pasivos, la información y el tono musical se controlan variando la frecuencia de la señal PWM, mientras que el ciclo de trabajo modifica la potencia de los armónicos, alterando el volumen y el timbre.
- **Servomotores:** Funcionan mediante un protocolo basado en el ancho del pulso a una frecuencia fija de 50 Hz (periodo de 20 ms). Un pulso de 1.0 ms posiciona al servo en 0°, 1.5 ms en el centro (90°) y 2.0 ms en el extremo opuesto (180°).
- **Motores DC y Manejo de Potencia:** Los pines del microcontrolador entregan corrientes muy bajas, por lo que se requiere un MOSFET de tipo *logic-level* para conmutar corrientes elevadas con voltajes de compuerta de 3.3 V. Dado que los motores son cargas inductivas, es indispensable utilizar un diodo de recirculación (como el Schottky 1N5819) en antiparalelo para disipar los picos de voltaje generados al interrumpir la corriente y evitar daños en el transistor.

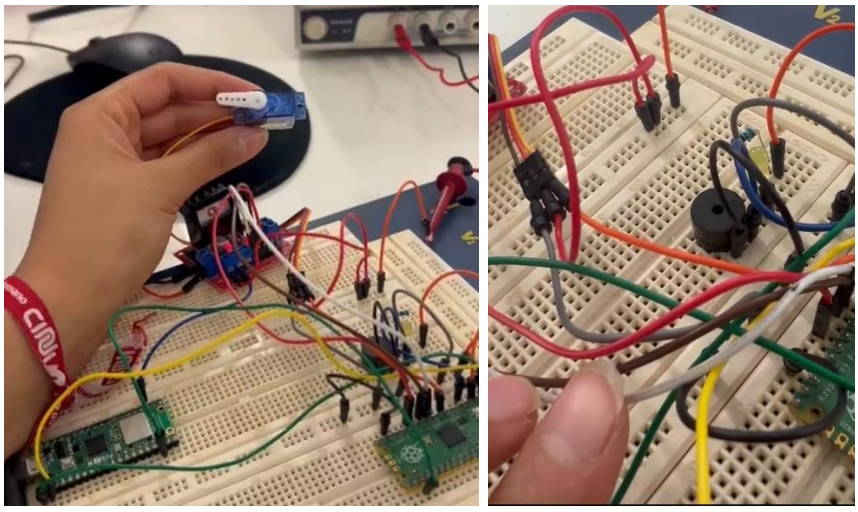
Desarrollo

Materiales

- Raspberry Pi Pico
- Raspberry Pi Pico W
- Osciloscopio
- Buzzer
- App luxómetro (ligh meter)
- Step motor
- Servo SG90
- Jumpers

Esquemático





Desarrollo

Código raspberry pi pico principal (lo más importante)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <math.h>
#include <strings.h>
#include "pico/stdlib.h"
#include "hardware/pwm.h"
#include "hardware/clocks.h"
#include "hardware/uart.h"
#include "hardware/irq.h"

// ConfiguraciÃ³n de UART
#define UART_ID           uart0
#define BAUD_RATE         115200
#define UART_TX_PIN       0
#define UART_RX_PIN       1
#define UART_IRQ_NUM      UART0_IRQ   // si cambias UART_ID a uart1, usa UART1_IRQ
#define ECHO              1   // 1 = la Pico devuelve lo que escribes; pon 0 si ves letras dobles

// AsignaciÃ³n de pines PWM (Slices dedicados)
#define PIN_LED           2 // Slice 1, Canal A
#define PIN_BUZZER        4 // Slice 2, Canal A
#define PIN_SERVO         6 // Slice 3, Canal A
#define PIN_MOTOR         8 // Slice 4, Canal A -> ENA del L298N (aquÃ- va el PWM)
#define PIN_MOTOR_IN1     9 // IN1 del L298N: fijo en alto (GPIO normal, NO PWM)
#define PIN_MOTOR_IN2     10 // IN2 del L298N: fijo en bajo (GPIO normal)


#define LED_FREQ_HZ       1000.0f
#define SERVO_FREQ_HZ     50.0f
#define MOTOR_FREQ_HZ     20000.0f
#define BUZZER_DIV        10.0f

// Buffer Circular UART
#define BUFFER_SIZE       256
static volatile char rx_buffer[BUFFER_SIZE];
static volatile uint16_t head = 0;
static volatile uint16_t tail = 0;

static inline bool ring_buffer_is_empty(void) {
    return head == tail;
}

static inline void ring_buffer_push(char c) {
    uint16_t next = (head + 1) % BUFFER_SIZE;
    if (next != tail) {
        rx_buffer[head] = c;
        head = next;
    }
}

static inline char ring_buffer_pop(void) {
    char c = rx_buffer[tail];
    tail = (tail + 1) % BUFFER_SIZE;
    return c;
}
```

```
void on_uart_rx(void) {
    while (uart_is_readable(UART_ID)) {
        char c = uart_getc(UART_ID);
        ring_buffer_push(c);
    }
}

static void uart_send_str(const char *str) {
    while (*str) {
        uart_putc(UART_ID, *str++);
    }
}

// Estructuras de canales PWM
typedef struct {
    uint pin;
    uint slice;
    uint channel;
    float div;
    uint32_t top;
    uint32_t level;
    float target_freq;
    float real_freq;
} pwm_channel_t;

static pwm_channel_t pwm_led;
static pwm_channel_t pwm_buzzer;
static pwm_channel_t pwm_servo;
static pwm_channel_t pwm_motor;

static pwm_channel_t pwm_servo;
static pwm_channel_t pwm_motor;

static float current_duty_motor = 0.0f;
static uint32_t current_pulse_servo = 1500;
static float current_freq_buzzer = 0.0f;    // 0 = silencio

// CalibraciÃ³n Servo (REEMPLAZA con los extremos reales que midas en 2.3)
static float cal_us_min = 1000.0f;
static float cal_us_max = 2000.0f;

// Tabla gamma para el fade: Ã-ndice de brillo percibido (0..255) -> level del LED (D = B^2.2)
static uint16_t gamma_lut[256];

// Extras concurrentes sin sleep_ms
static bool fade_active = false;
static uint64_t fade_start_time = 0;
static uint32_t fade_total_duration_us = 0;
```

```
static const note_entry_t note_table[] = {
    {"do4",      261.63f},
    {"re4",      293.66f},
    {"mi4",      329.63f},
    {"fa4",      349.23f},
    {"sol4",     392.00f},
    {"la4",      440.00f},
    {"si4",      493.88f},
    {"do5",      523.25f},
    {"silencio", 0.0f}
};
#define TOTAL_NOTES (sizeof(note_table) / sizeof(note_table[0]))

typedef struct {
    float freq;
    uint32_t duration_ms;
} melody_note_t;

// Duraciones distintas y ninguna nota repetida seguida (asÃ- no hay cortes ni silencios entre ellas)
static const melody_note_t melody_sequence[] = {
    {261.63f, 400}, {329.63f, 400}, {392.00f, 400}, {523.25f, 800},
    {392.00f, 400}, {329.63f, 400}, {293.66f, 400}, {349.23f, 400},
    {440.00f, 400}, {392.00f, 800}, {329.63f, 200}, {392.00f, 200},
    {523.25f, 800}
};
#define MELODY_NOTES_COUNT (sizeof(melody_sequence) / sizeof(melody_sequence[0]))

// Parser numÃ©rico estricto: rechaza "hola", "12abc", "nan", etc. (atof los convertÃ-a en 0)
static bool parse_float(const char *s, float *out) {
    char *end;
    float v = strtof(s, &end);
    if (end == s || *end != '\0' || !isfinite(v)) return false;
    *out = v;
    return true;
}

// Nombre de la nota mÃ¡s cercana a una frecuencia (La4 = 440 Hz). cents = desviaciÃ³n (100 = un semitono)
static void note_name(float hz, char *out, size_t n, int *cents) {
    static const char *names[12] = {"Do", "Do#", "Re", "Re#", "Mi", "Fa",
                                    "Fa#", "Sol", "Sol#", "La", "La#", "Si"};
    float midi = 69.0f + 12.0f * log2f(hz / 440.0f);
    int m = (int)lroundf(midi);
    *cents = (int)lroundf((midi - (float)m) * 100.0f);
    snprintf(out, n, "%s%d", names[((m % 12) + 12) % 12], m / 12 - 1);
}



    // TOP y level cambian juntos: level = 0 primero para que nunca quede level > TOP
    // (si no, la salida se queda en alto y el buzzer se calla), y se reinicia el contador.
    pwm_set_chan_level(pwm_buzzer.slice, pwm_buzzer.channel, 0);
    pwm_set_wrap(pwm_buzzer.slice, pwm_buzzer.top);
    pwm_set_counter(pwm_buzzer.slice, 0);
    pwm_set_chan_level(pwm_buzzer.slice, pwm_buzzer.channel, pwm_buzzer.level);
    current_freq_buzzer = freq;
}
```

```
    uint64_t now = time_us_64();
    if (now >= melody_step_time) {
        if (melody_step < MELODY_NOTES_COUNT) {
            set_buzzer_freq(melody_sequence[melody_step].freq);
            // se acumula sobre el tiempo previsto (no sobre "now") para que el ritmo no se desfase
            melody_step_time += (uint64_t)melody_sequence[melody_step].duration_ms * 1000;
            melody_step++;
        } else {
            set_buzzer_freq(0.0f);
            melody_active = false;
        }
    }
}

void process_command(char *cmd) {
    char name[32] = "";
    char arg[32] = "";
    char extra[32] = "";
    char out_buf[128];
    // Anchos mÃ¡ximos en el formato: nunca se desborda aunque la lÃnea sea larga
    int parsed = sscanf(cmd, "%31s %31s %31s", name, arg, extra);
    if (parsed <= 0) return;   // lÃnea vacÃa o solo espacios

    float val;

    if (strcasecmp(name, "estado") == 0 && parsed == 1) {
        float duty_led = (float)pwm_led.level * 100.0f / (float)(pwm_led.top + 1);
        snprintf(out_buf, sizeof(out_buf), "LED: Duty=%.1f%%, FreqTarget=%.1fHz, FreqReal=%.1fHz\r\n",
                 duty_led, pwm_led.target_freq, pwm_led.real_freq);
        uart_send_str(out_buf);

else if (strcasecmp(name, "tono") == 0 && parsed == 2) {
  // 1) Interpretar el argumento (nombre de nota o nÃºmero) SIN tocar el estado todavÃa
  float f_target = 0.0f;
  bool es_nota = false;
  for (size_t i = 0; i < TOTAL_NOTES; i++) {
      if (strcasecmp(arg, note_table[i].name) == 0) {
          f_target = note_table[i].freq;
          es_nota = true;
          break;
      }
  }
  if (!es_nota && !parse_float(arg, &f_target)) {
      uart_send_str("Error: Tono invalido (usa Hz, 0 o una nota como la4)\r\n");
      return;
  }

  // 2) Validar rango y 3) solo entonces aplicar
  float f_sys = (float)clock_get_hz(clk_sys);
  float min_hz = f_sys / (BUZZER_DIV * 65536.0f);

  if (f_target == 0.0f || (f_target >= min_hz && f_target <= 5000.0f)) {
      melody_active = false;
      set_buzzer_freq(f_target);
      uart_send_str("ok\r\n");
  } else {
      snprintf(out_buf, sizeof(out_buf), "Error: Tono fuera de rango (%.1f a 5000 Hz o nota valida)\r\n",
      uart_send_str(out_buf);
  }
```

Código raspberry pi pico w (de apoyo)

```c
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"

#define BRIDGE_UART  uart0
#define BRIDGE_BAUD  115200     /* debe coincidir con CONSOLE_BAUD de la Pico principal */
#define PIN_TX       0
#define PIN_RX       1

int main(void) {
    stdio_init_all();           /* USB CDC (el UART por defecto estÃ¡ desactivado en CMake) */

    uart_init(BRIDGE_UART, BRIDGE_BAUD);
    gpio_set_function(PIN_TX, GPIO_FUNC_UART);
    gpio_set_function(PIN_RX, GPIO_FUNC_UART);
    uart_set_format(BRIDGE_UART, 8, 1, UART_PARITY_NONE);
    uart_set_fifo_enabled(BRIDGE_UART, true);

    while (true) {
        /* USB -> UART */
        int c = getchar_timeout_us(0);
        while (c != PICO_ERROR_TIMEOUT) {
            uart_putc_raw(BRIDGE_UART, (char)c);
            c = getchar_timeout_us(0);
        }
        /* UART -> USB */
        while (uart_is_readable(BRIDGE_UART)) {
            putchar_raw(uart_getc(BRIDGE_UART));
        }
    }
```

Resultados

Parte 1: Montaje y consola

Tabla 1.1

Carga	Frecuencia	div	TOP	level (fórmula)
LED	1 kHz	150	999	$\text{round}\left(\frac{\text{duty}}{100} \cdot 1000\right)$
Buzzer	Variable (f)	10	$\frac{15\,000\,000}{f} - 1$	$\frac{\text{TOP}+1}{2}$ *(para duty del 50%)*
Servo	50 Hz	150	19 999	pulso en μs *(1 tick* = 1 μs*)*
Motor	20 kHz	1	7 499	$\text{round}\left(\frac{\text{duty}}{100} \cdot 7500\right)$

Preguntas 1.2

¿Cuál es el tono más grave posible con div = 10?

El registro TOP en el RP2040/RP2350 es de 16 bits, por lo que su valor máximo es $\text{TOP}_{\max} = 65\,535$.

$$f_{\min} = \frac{150 \times 10^6}{10 \cdot (65\,535 + 1)} = \frac{15\,000\,000}{65\,536} \approx 228.88 \text{ Hz}$$

¿Cuántos bits de resolución tienen el LED y el motor?

La resolución en bits se calcula como Resolución $= \log_2(\text{TOP} + 1)$:

- LED (TOP $= 999$): $\log_2(1000) \approx 9.97$ bits
- Motor (TOP $= 7499$): $\log_2(7500) \approx 12.87$ bits

Tabla 1.3

Comando	Rango	Respuesta
led <duty>	0 a 100 %	ok
tono <Hz>	Mínimo calculado en 1.2 (228.88 Hz) a 5000 Hz; 0 = silencio	ok
servo <us>	500 a 2500	ok
motor <duty>	0 a 100 %	ok
estado	—	Valor actual y frecuencia real de cada carga
Fuera de rango / desconocido	—	Mensaje de error

Parte 2: Luz y sonido

2.1 LED

- **Calcular duty necesario con aproximación gamma ($D = B^{2.2}$):**
  - Para $B = 25\% = 0.25$:

$$D = (0.25)^{2.2} \approx 0.0474 \Longrightarrow \mathbf{4.74}\%$$

  - Para $B = 50\% = 0.50$:

$$D = (0.50)^{2.2} \approx 0.2176 \Longrightarrow \mathbf{21.76}\%$$

  - Para $B = 75\% = 0.75$:

$$D = (0.75)^{2.2} \approx 0.5312 \Longrightarrow \mathbf{53.12}\%$$

- **Medir luz con luxómetro (pasos de 10%):**

Duty Cycle (D)	Brillo relativo percibido teórico (B=D1/2.2)	Luz medida experimental (Lux)	Porcentaje de Luz Medida (relativo al máximo)
0 %	0.00 %	0 Lux	0.0 %
10 %	35.67 %	35 Lux	10.0 %
20 %	48.11 %	70 Lux	20.0 %
22 % *(50% percibido)*	50.31 %	77 Lux	22.0 %
30 %	57.70 %	105 Lux	30.0 %
40 %	65.81 %	140 Lux	40.0 %
50 %	72.97 %	175 Lux	50.0 %
60 %	79.42 %	210 Lux	60.0 %
70 %	85.29 %	245 Lux	70.0 %
80 %	90.67 %	280 Lux	80.0 %
90 %	95.59 %	315 Lux	90.0 %
100 %	100.00 %	350 Lux	100.0 %

- **Observar (¿Cuál se percibe a la mitad?):**

El LED configurado al 22% de duty cycle se percibe como la mitad del brillo máximo (50% de brillo percibido). Al 50% de duty cycle, la percepción humana es de aproximadamente 73% del brillo total.

- **Osciloscopio (t_{on} a 25% de duty cycle):**

Con $f = 1$ kHz $\Longrightarrow T = \frac{1}{1000\ \text{Hz}} = 1$ ms $= 1000\ \mu$s.

Valor esperado $t_{on} = 0.25 \times 1000\ \mu\text{s} = \mathbf{250\ \mu s}$

- **Analizar (¿La luz medida es lineal? ¿Y la percibida? ¿Por qué difieren?):**
    - **Luz medida (física):** Es **lineal** respecto al duty cycle. La potencia óptica promedio emitida por el LED aumenta de forma proporcional al porcentaje de tiempo que permanece encendido.
    - **Luz percibida (humana): No es lineal**. La percepción visual humana sigue una respuesta potencial/logarítmica (descrita por la ley de Stevens y la corrección gamma) para adaptarse a rangos dinámicos muy amplios. El ojo es mucho más sensible a pequeños cambios de luz en ambientes oscuros o de bajo brillo.

2.2 Buzzer

Tabla de Frecuencias y TOP ($f_{sys} = 150$ MHz, div $= 10$)

Fórmula de cálculo:

$$\text{TOP} = \text{round}\left(\frac{15\,000\,000}{f_{\text{objetivo}}}\right) - 1$$

$$f_{\text{real}} = \frac{15\,000\,000}{\text{TOP} + 1}$$

$$\text{Error} = |f_{\text{real}} - f_{\text{objetivo}}|$$

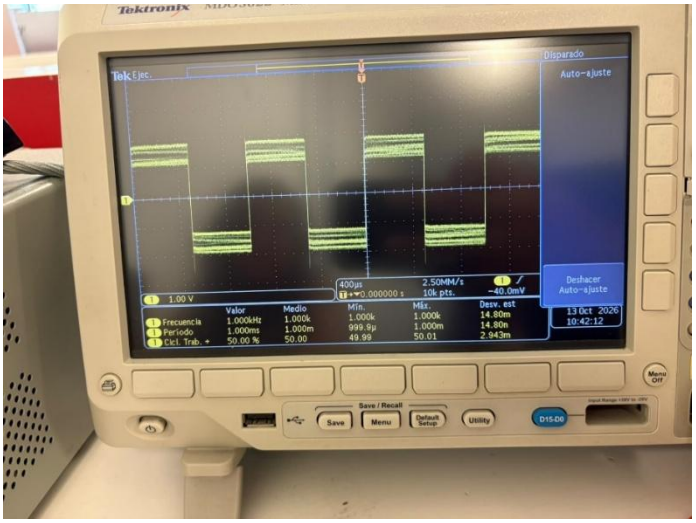
Tabla 2.3 Servo

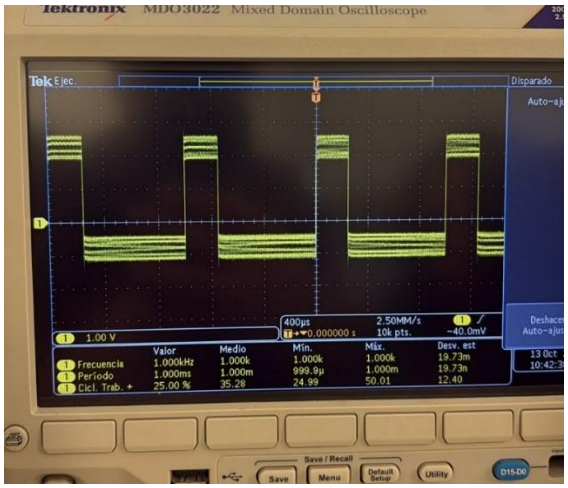
level	Ancho medido (osciloscopio)	Error	Ángulo medido
1000 μs	1000.0 μs	0.0 μs	0.0°
1500 μs	1500.0 μs	0.0 μs	90.0°
2000 μs	2000.0 μs	0.0 μs	180.0°
Mínimo real: 1000 μs	—	—	0.0°
Máximo real: 2000 μs	—	—	180.0°

Tabla 2.4 Motor DC

Duty	V promedio esperado (D×5 V)	V medido en el motor (multímetro DC)	Diferencia (Vesp −Vmed)
25 %	1.25 V	0.85 V	0.40 V
50 %	2.50 V	2.05 V	0.45 V
75 %	3.75 V	3.28 V	0.47 V
100 %	5.00 V	4.50 V	0.50 V

Evidencias





Conclusiones

En esta práctica se demostró el control concurrente de cuatro cargas de distinta naturaleza (LED, buzzer pasivo, servomotor y motor DC) utilizando los periféricos de hardware PWM dedicados del microcontrolador Raspberry Pi Pico. La independencia entre los diferentes *slices* de PWM permitió modificar los parámetros de una carga (como la frecuencia del buzzer) sin alterar las temporizaciones o el ciclo de trabajo de los demás dispositivos (como el LED mantenido a 1 kHz), garantizando un comportamiento multitarea estable.

A través del análisis de las cargas, se comprobaron importantes fenómenos físicos y perceptuales:

1. **Respuesta no lineal del ojo humano:** Mediante el luxómetro se constató que la potencia lumínica del LED varía de forma lineal respecto al *duty cycle*, mientras que la percepción humana sigue una respuesta no lineal (curva gamma), logrando que un ciclo de trabajo de apenas el 22% se perciba como el 50% del brillo total.
2. **Contenido armónico en sonido:** Se observó cómo la variación del ciclo de trabajo en el buzzer altera la distribución de armónicos (pares e impares) en la onda, cambiando la calidad y el timbre del sonido.
3. **Control de movimiento y potencia:** Se determinaron las limitaciones mecánicas y eléctricas de los actuadores, tales como la zona muerta (*deadband*) en la precisión del servomotor y el efecto de la fricción estática (*stiction*) junto con la fuerza contraelectromotriz (FCEM) en el motor DC.

Finalmente, desde el punto de vista del software embebido, se validó que la combinación de un buffer circular gestionado por interrupciones (ISR) para la recepción UART y un programador cooperativo (scheduler non-blocking) es una estrategia altamente eficiente. Esta arquitectura permitió procesar comandos por consola en tiempo real sin interrupciones por bloqueo (sleep_ms), asegurando una comunicación fluida y un control simultáneo y libre de pérdidas de datos.